

AI-NATIVE GROCERY MERCHANT

Document 08 — API Contract Specification

Development-only specification • Locked MVP • REST contracts, payloads, validation, errors and service boundaries

API principle: The API exposes controlled capabilities, not internal authority. The client and AI can request actions; authoritative backend services validate, calculate and authorize them.

Document status: LOCKED DEVELOPMENT SPECIFICATION

Primary style: REST/JSON over HTTPS with versioned routes, typed request/response schemas and deterministic error codes.

1. Purpose and Scope

This document defines the API contracts for the locked AI-Native Grocery Merchant MVP. It maps the functional workflow into explicit HTTP endpoints, request/response payloads, validation rules, authorization boundaries and error semantics.

The API is designed for a modular monolith. Internal modules may call service functions directly, but externally visible behavior should follow the same contracts and validation rules.

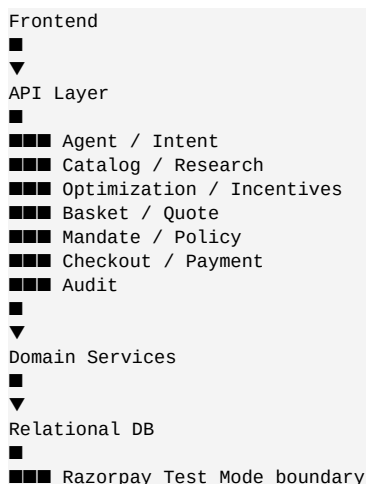
1.1 In scope

- Shopping intent creation and requirement extraction.
- Catalog and product search.
- Quality evidence retrieval.
- Optimization execution and basket retrieval.
- Incentive evaluation.
- User basket selection.
- Authoritative basket quoting.
- Mandate retrieval and policy evaluation.
- Checkout initiation after policy ALLOW.
- Payment status and webhook ingestion.
- Audit retrieval for the current workflow.

1.2 Out of scope

- Public marketplace APIs.
- External unrestricted product search.
- Production payment APIs.
- Refund/payout APIs.
- Full loyalty platform APIs.
- Multi-agent orchestration APIs.

2. API Architecture



The AI agent should normally access the same backend domain capabilities through controlled server-side tool adapters rather than exposing privileged HTTP credentials to the model.

3. Common API Conventions

3.1 Base path

```
/api/v1
```

3.2 Content type

```
Content-Type: application/json
Accept: application/json
```

3.3 Identifier convention

Resource identifiers use UUIDs. External Razorpay identifiers remain strings.

3.4 Money convention

All money fields in API payloads are integer minor units. For INR, ₹874 is represented as 87400. APIs may additionally return formatted display values, but those are non-authoritative.

3.5 Timestamps

API timestamps use ISO 8601 UTC strings.

3.6 Correlation

```
X-Request-ID: req-7d9...
Idempotency-Key: idem-...
```

Request IDs should be accepted or generated for tracing. Idempotency keys are required on operations that can create or authorize financial state.

4. Standard Response Envelope

4.1 Success

```
{
  "data": {},
  "meta": {
    "request_id": "req-001"
  }
}
```

4.2 Error

```
{
  "error": {
    "code": "MAX_SPEND_EXCEEDED",
    "message": "Final payable exceeds the authorized maximum spend.",
    "details": {},
    "request_id": "req-001"
  }
}
```

Client-facing messages should be safe and useful. Internal stack traces, secrets and privileged implementation details must not be returned.

5. Authentication and Authorization

All non-public application APIs require authenticated user/session context. Internal service-to-service operations require server authentication/authorization.

5.1 Authority rules

- Authentication identifies the caller; it does not grant mandate authority.
- Mandate authorization is evaluated by the policy engine.
- Frontend requests are never treated as trusted financial instructions.
- AI tool calls are executed through backend adapters with scoped permissions.
- Payment credentials and Razorpay secrets remain server-side.

5.2 Context binding

Resource identifiers such as `mandate_id`, `session_id`, `basket_id` and `order_id` must be checked against the authenticated context. A valid UUID from another user's session must return an authorization error rather than leaking resource data.

6. Endpoint Inventory

Method	Endpoint	Purpose
POST	/sessions	Create shopping session
POST	/sessions/{session_id}/intents	Create shopping intent
POST	/intents/{intent_id}/requirements:extract	Extract/validate requirements
GET	/catalog/search	Search controlled catalog
GET	/products/{product_id}	Get product details
GET	/products/{product_id}/evidence	Get quality evidence
POST	/optimization-runs	Run optimization
GET	/optimization-runs/{id}	Get optimization result
POST	/optimization-runs/{id}/incentives:evaluate	Evaluate incentives
POST	/sessions/{id}/basket-selections	Select basket
POST	/baskets/{id}/quote	Create fresh authoritative quote
GET	/mandates/{id}	Get mandate
POST	/policy/evaluate	Evaluate authorization
POST	/checkout	Create checkout/payment order after ALLOW
GET	/orders/{id}	Get order status
GET	/payments/{id}	Get payment status
POST	/webhooks/razorpay	Receive Razorpay webhook
GET	/sessions/{id}/audit	Get workflow audit trail

7. Session API

7.1 POST /sessions

Creates a shopping workflow session.

```
Request
{
  "currency": "INR"
}

Response
{
  "data": {
    "session_id": "session-001",
    "status": "ACTIVE",
    "currency": "INR"
  },
  "meta": {
    "request_id": "req-001"
  }
}
```

7.2 Validation

- Currency must be supported.
- Authenticated user context is required.
- Session starts in ACTIVE state.

8. Shopping Intent API

8.1 POST /sessions/{session_id}/intents

```
Request
{
  "goal_text": "Pasta for 4, budget ■1000",
  "budget_minor": 100000,
  "category": "grocery",
  "quality_preference": null,
  "constraints": []
}

Response
{
  "data": {
    "intent_id": "intent-001",
    "session_id": "session-001",
    "mandate_id": "mandate-001",
    "status": "CREATED"
  }
}
```

The budget is the user's stated planning budget. The mandate remains the authoritative authorization ceiling.

8.2 POST /intents/{intent_id}/requirements:extract

```
Response
{
  "data": {
    "intent_id": "intent-001",
    "requirements": [
      {
        "requirement_id": "req-001",
        "item_name": "pasta",
        "target_quantity": 2,
        "unit": "packs",
        "minimum_quality": "acceptable",
        "confidence": 0.94
      }
    ],
    "assumptions": []
  }
}
```

If material ambiguity prevents safe planning, the API should return a structured clarification state rather than inventing critical requirements.

9. Catalog APIs

9.1 GET /catalog/search

```
Query
?q=eggs&category=grocery&limit=20

Response
{
  "data": {
    "items": [
      {
        "product_id": "prod-001",
        "sku_id": "sku-001",
        "name": "Farm Eggs",
        "pack_quantity": 6,
        "pack_unit": "pieces",
        "price_minor": 3600,
        "currency": "INR",
        "stock_available": true
      }
    ]
  }
}
```

Search is controlled and bounded. The API must not expose arbitrary database queries.

9.2 GET /products/{product_id}

Returns authoritative product metadata, available SKUs and current catalog state permitted for the authenticated workflow.

9.3 GET /products/{product_id}/evidence

```
Response
{
  "data": {
    "evidence": [
      {
        "evidence_id": "evidence-21",
        "source_type": "SEEDED_REVIEW_SUMMARY",
        "summary": "Consistently positive quality signals.",
        "quality_signal": "GOOD",
        "confidence": 0.86
      }
    ]
  }
}
```

Evidence is informational for optimization; it does not override authoritative product/stock data.

10. Optimization APIs

10.1 POST /optimization-runs

```
Request
{
  "intent_id": "intent-001",
  "mandate_id": "mandate-001"
}

Response
{
  "data": {
    "optimization_run_id": "opt-001",
    "status": "COMPLETED",
    "best_value_basket_id": "basket-value-001",
    "best_quality_basket_id": "basket-quality-001",
    "recommendation": {
      "basket_id": "basket-value-001",
      "reason": "Lower final payable with acceptable quality."
    }
  }
}
```

The optimization service may internally execute several controlled tool operations. The public API should return the resulting structured state rather than exposing raw model context.

10.2 GET /optimization-runs/{id}

Returns requirements, basket references, recommendation, incentive decisions and relevant decision metadata for the current workflow.

11. Incentive API

11.1 POST /optimization-runs/{id}/incentives:evaluate

```
Request
{
  "basket_ids": [
    "basket-value-001",
    "basket-quality-001"
  ]
}

Response
{
  "data": {
    "evaluations": [
      {
        "basket_id": "basket-value-001",
        "incentive_id": "voucher-001",
        "decision": "SAVE_FOR_LATER",
        "current_saving_minor": 500,
        "reason": "Current saving is small relative to expected future utility."
      }
    ]
  }
}
```

```
]
}
```

The incentive service determines eligibility and actual saving. The AI may reason about whether the benefit is worthwhile but cannot fabricate eligibility or discount amounts.

12. Basket Selection API

12.1 POST /sessions/{session_id}/basket-selections

```
Request
{
  "basket_id": "basket-quality-001",
  "selection_source": "USER"
}
```

```
Response
{
  "data": {
    "selection_id": "selection-001",
    "basket_id": "basket-quality-001",
    "status": "SELECTED"
  }
}
```

This operation records user preference. It does not authorize payment.

12.2 Rules

- Basket must belong to the session.
- Basket must be a valid current optimization result.
- A new selection supersedes the prior selection.
- Selection must not create an order automatically.

13. Quote API

13.1 POST /baskets/{id}/quote

Creates a fresh authoritative quote immediately before policy evaluation.

```
Response
{
  "data": {
    "basket_id": "basket-quality-001",
    "gross_amount_minor": 92000,
    "discount_amount_minor": 4600,
    "final_payable_minor": 87400,
    "currency": "INR",
    "quote_version": "quote-009"
  }
}
```

13.2 Quote rules

- Re-read current prices.
- Re-read stock.
- Validate applicable incentives.
- Recalculate line amounts and totals.
- Never trust client totals.
- Quote version must be traceable to the authorization attempt.

14. Mandate API

14.1 GET /mandates/{id}

```
Response
{
  "data": {
    "mandate_id": "mandate-001",
    "max_spend_minor": 100000,
    "currency": "INR",
    "allowed_categories": ["grocery"],
    "max_per_item_minor": 30000,
    "valid_until": "2026-09-07T23:59:59Z",
    "status": "ACTIVE",
    "purpose": "Pasta ingredients for 4"
  }
}
```

Mandate retrieval is read-only for the shopping agent. No agent endpoint is provided for modifying mandate authority.

15. Policy Evaluation API

15.1 POST /policy/evaluate

This is the central authorization contract.

```
Request
{
  "mandate_id": "mandate-001",
  "basket_id": "basket-quality-001",
  "quote_version": "quote-009"
}

Response - ALLOW
{
  "data": {
    "policy_decision_id": "pd-001",
    "decision": "ALLOW",
    "reason_code": "AUTHORIZED",
    "final_payable_minor": 87400,
    "policy_version": "policy-v1"
  }
}
```

15.2 DENY example

```
Response
{
  "error": {
    "code": "MAX_SPEND_EXCEEDED",
    "message": "Final payable exceeds the authorized maximum spend.",
    "details": {
      "final_payable_minor": 85000,
      "max_spend_minor": 80000,
      "recoverable": true
    },
    "request_id": "req-018"
  }
}
```

15.3 Policy rules

The API must enforce the ordered checks specified in Document 06: mandate validity → category → stock → maximum per item → incentive validity → authoritative amount calculation → maximum spend → decision → audit.

A successful response is scoped to the exact mandate, basket and quote context.

16. Checkout API

16.1 POST /checkout

Creates the Razorpay Test Mode order only after successful policy authorization.

```
Request
{
  "selection_id": "selection-001",
  "policy_decision_id": "pd-001"
}
```

```
Response
{
  "data": {
    "order_id": "order-001",
    "razorpay_order_id": "order_test_123",
    "amount_minor": 87400,
    "currency": "INR",
    "status": "PAYMENT_PENDING"
  }
}
```

16.2 Server-side requirements

- Validate selection ownership.
- Validate policy decision ownership.
- Require policy decision = ALLOW.
- Confirm policy decision is current and not stale.
- Use the authorized amount, never a client-supplied amount.
- Create Razorpay order in Test Mode.
- Persist the external order reference.
- Use idempotency to prevent duplicate order creation.

The checkout endpoint is a financial boundary and must fail closed if authorization cannot be established.

17. Order API

17.1 GET /orders/{id}

```
Response
{
  "data": {
    "order_id": "order-001",
    "status": "PAYMENT_PENDING",
    "amount_minor": 87400,
    "currency": "INR",
    "razorpay_order_id": "order_test_123",
    "payment_id": null
  }
}
```

Order status is server-derived. The frontend may display it but cannot mutate it directly.

18. Payment API

18.1 GET /payments/{id}

```
Response
{
  "data": {
    "payment_id": "payment-001",
    "order_id": "order-001",
    "status": "VERIFIED",
    "amount_minor": 87400,
    "currency": "INR",
    "razorpay_payment_id": "pay_test_123",
    "verified_at": "2026-09-02T12:04:00Z"
  }
}
```

Payment success must be derived from backend verification and/or validated Razorpay webhook state. A client callback or AI response cannot establish payment success.

19. Razorpay Webhook API

19.1 POST /webhooks/razorpay

Receives Razorpay webhook events at the server boundary. Signature validation occurs before event processing.

```
Headers:
X-Razorpay-Signature: <signature>

Body:
{
  "event": "...",
  "payload": { ... }
}
```

19.2 Processing rules

- Validate webhook signature.
- Identify event and external payment/order reference.
- Check duplicate/event idempotency state.
- Update order/payment state through the backend state machine.
- Record audit event.
- Return success only after safe processing/acceptance.
- Never let webhook payload directly set arbitrary application state.

Webhook-specific implementation details belong to Document 09.

20. Audit API

20.1 GET /sessions/{session_id}/audit

```
Response
{
  "data": {
    "events": [
      {
        "audit_event_id": "audit-001",
        "event_type": "BASKET_RECOMMENDED",
        "occurred_at": "2026-09-02T12:00:00Z",
        "event_data": {
          "basket_id": "basket-value-001",
          "reason": "Lower final payable with acceptable quality."
        }
      },
      {
        "audit_event_id": "audit-002",
        "event_type": "POLICY_DECISION",
        "occurred_at": "2026-09-02T12:02:00Z",
        "event_data": {
          "decision": "ALLOW",
          "policy_version": "policy-v1"
        }
      }
    ]
  }
}
```

Audit output should be sufficient to trace the money path without exposing secrets or hidden chain-of-thought.

21. AI Tool Adapter Contracts

The AI agent should not receive unrestricted API access. Each model tool maps to one narrow backend capability.

AI tool	Backend endpoint/service	Allowed purpose
search_catalog	GET /catalog/search	Find candidates
get_product	GET /products/{id}	Read product facts
get_quality_evidence	GET /products/{id}/evidence	Read evidence
run_optimization	POST /optimization-runs	Request bounded optimization
evaluate_incentives	POST /optimization-runs/{id}/incentives:evaluate	Evaluate incentive options

AI tool	Backend endpoint/service	Allowed purpose
get_quote	POST /baskets/{id}/quote	Read authoritative quote
evaluate_policy	POST /policy/evaluate	Request authorization
record_audit	Internal audit service	Record decision metadata

No AI tool is provided for mandate modification, direct Razorpay order creation, payment authorization or arbitrary database access.

22. Validation Rules

Input	Validation
session_id	Exists and belongs to authenticated user
intent_id	Belongs to session/user and is in valid workflow state
mandate_id	Belongs to user/context and is active for authorization
basket_id	Belongs to optimization/session and is current
sku_id	Exists, allowed and available
quantity	Positive and within domain constraints
amount_minor	Server-derived for financial operations
quote_version	Matches current quote context
policy_decision_id	Exists, belongs to basket/mandate and is ALLOW
selection_id	Belongs to session and selected by user
idempotency key	Required on financial creation operations

23. Error Model

HTTP	Code	Meaning
400	VALIDATION_ERROR	Malformed or invalid request
401	UNAUTHENTICATED	Authentication required/invalid
403	FORBIDDEN	Authenticated but not authorized for resource
404	NOT_FOUND	Resource does not exist in caller context
409	STATE_CONFLICT	Operation conflicts with current workflow state
409	IDEMPOTENCY_CONFLICT	Same key used with incompatible request
422	MANDATE_INVALID	Authorization context cannot be used
422	CATEGORY_NOT_ALLOWED	Basket violates category rule
422	STOCK_UNAVAILABLE	Required stock unavailable
422	MAX_PER_ITEM_EXCEEDED	Item violates mandate limit
422	INCENTIVE_INVALID	Incentive cannot be applied
422	MAX_SPEND_EXCEEDED	Final payable exceeds mandate
424	PAYMENT_PROVIDER_ERROR	Razorpay boundary failed
500	INTERNAL_ERROR	Unexpected server failure

24. Idempotency

24.1 Required operations

- POST /policy/evaluate should support idempotent authorization requests.

- POST /checkout must be idempotent.
- Webhook processing must be idempotent.
- Any operation that can create an order/payment state must have a stable deduplication strategy.

24.2 Example

```
POST /checkout
Idempotency-Key: checkout-selection-001

Repeated request
→ returns the same application order/payment-order result
→ does not create a second Razorpay order
```

25. State and Concurrency Rules

APIs must validate workflow state before mutation.

Operation	Required state
Create intent	Session ACTIVE
Extract requirements	Intent CREATED/PLANNING
Optimize	Intent requirements available; mandate usable
Select basket	Optimization completed
Quote	Basket valid/current
Policy evaluate	Fresh quote available
Checkout	User selection + current ALLOW
Payment update	Order PAYMENT_PENDING or compatible provider state

Concurrent selections or checkout attempts must not create conflicting payment state. The backend should serialize or reject conflicting financial transitions.

26. Security Requirements

- HTTPS in all non-local environments.
- Authentication on user APIs.
- Server-to-server authentication for privileged service calls.
- Strict schema validation.
- Resource ownership checks.
- No arbitrary SQL/query endpoint.
- No client-controlled final amount.
- No client-controlled policy decision.
- No client-controlled payment status.
- Razorpay secrets only in server-side environment configuration.
- Webhook signature verification before state mutation.
- Sensitive data excluded from logs and audit payloads.

27. Observability and Correlation

Every workflow request should be traceable through a common request/session chain.

```
request_id
→ session_id
→ agent_run_id
→ optimization_run_id
```

```
→ basket_id
→ policy_decision_id
→ order_id
→ razorpay_order_id
→ payment_id / razorpay_payment_id
```

Logs should include identifiers and state transitions, not secrets or unrestricted user content.

28. Example End-to-End API Sequence

```
1. POST /sessions
2. POST /sessions/{session_id}/intents
3. POST /intents/{intent_id}/requirements:extract
4. GET /catalog/search
5. GET /products/{id}/evidence
6. POST /optimization-runs
7. POST /optimization-runs/{id}/incentives:evaluate
8. POST /sessions/{id}/basket-selections
9. POST /baskets/{id}/quote
10. POST /policy/evaluate
11. if DENY:
    re-optimize → new basket → quote → policy
12. if ALLOW:
    POST /checkout
13. Razorpay checkout
14. POST /webhooks/razorpay
15. GET /orders/{id}
16. GET /sessions/{id}/audit
```

29. Graceful Failure Contract

29.1 Policy denial

A MAX_SPEND_EXCEEDED response must contain enough structured data for the optimization layer to recover without changing the mandate.

29.2 Stale quote

```
{
  "error": {
    "code": "QUOTE_STALE",
    "message": "Basket pricing changed; a fresh quote is required.",
    "details": {
      "basket_id": "basket-001"
    }
  }
}
```

29.3 Payment failure

The API must return a payment/order state such as PAYMENT_FAILED or PAYMENT_PENDING rather than claiming success. The agent may explain the state but cannot override it.

30. API Acceptance Tests

Scenario	Expected API behavior
Valid session + intent	Creates intent
Cross-user basket access	403/404 without data leakage
Malformed quantity	400 VALIDATION_ERROR
Search unavailable SKU	Candidate excluded / empty result
Fresh quote	Authoritative totals returned
Client manipulates amount	Server ignores client amount
Expired mandate	422 MANDATE_INVALID
Policy max-spend breach	422 MAX_SPEND_EXCEEDED
Valid ALLOW	Checkout can proceed

Scenario	Expected API behavior
Checkout without ALLOW	Rejected
Duplicate checkout request	Same result; no duplicate order
Webhook replay	No duplicate payment transition
Payment verification failure	No VERIFIED state
Stale basket quote	Re-quote required
Audit retrieval	Complete trace visible for session

31. Definition of Done

- All locked MVP workflow endpoints exist or have equivalent internal service contracts.
- Request and response schemas are implemented and validated.
- Money uses integer minor units.
- Authenticated resource ownership is enforced.
- AI tool adapters expose only permitted capabilities.
- Mandate modification endpoints are not exposed to the agent.
- Policy evaluation is deterministic and server-side.
- Checkout requires policy ALLOW.
- Checkout amount is server-derived.
- Idempotency protects financial creation operations.
- Razorpay webhook processing is signature-validated and idempotent.
- Payment state cannot be set by frontend or AI.
- Structured denial codes support optimization recovery.
- Audit correlation identifiers are persisted.
- API error behavior matches the defined error model.
- All acceptance scenarios pass.
- No API for out-of-scope functionality is introduced.

32. Related Development Documents

Document	Relationship
01 — Development Master Spec	Locked product and engineering baseline
02 — Functional Requirements	Functional behavior exposed through APIs
03 — System Architecture	API/service boundaries and trust model
04 — AI Agent Specification	Agent tool usage and output contracts
05 — Optimization Decision Specification	Optimization service behavior
06 — Policy & Mandate Specification	Authorization API behavior and reason codes
07 — Data Model Specification	Persistent entities represented by these APIs
09 — Razorpay Integration Specification	Payment/order/webhook implementation details
10 — Testing & Acceptance Specification	API and workflow test coverage